

ReDoS & SQL injection analysis with CodeQL

DD2525 Language-Based Security

Jonathan Nilsson Cullgert jonnc@kth.se
Rasmus Sjöberg rassjo@kth.se

June 2026

1 Introduction

This project focuses on exploring Regular Expression Denial of Service (ReDoS) attacks and SQL injection attacks. More specifically, the project focuses on identifying these vulnerabilities in open-source projects on GitHub. This will be done using a tool called CodeQL, which identifies security weaknesses by querying a database representation of the source code.

As mentioned above, the project is focused on two security vulnerabilities. The first one, ReDoS, was introduced in the lab in web application security. The second one, SQL injection, is another web security vulnerability, in which untrusted input is incorporated into database queries in an unsafe manner. To find these vulnerabilities, two queries are created and run against 50 popular and 50 unpopular open-source projects each. The results are then analysed to determine whether the reported findings correspond to real ReDoS or SQL injection vulnerabilities, or whether they are false positives.

2 Background

In this chapter, concepts relevant to this project are introduced to provide an essential background.

2.1 RegEx

Regular Expression, or RegEx, is a way in code to search for patterns of substrings inside a string. This is commonly used in applications since it can be a way to, for example, verify that an input is a valid email address, ensure that a password meets certain constraints, or just in general, validate user input.

Regular expressions can be created using an enclosed backslash pattern or by using the `RegExp` constructor. These expressions can also be found in the `RegExp` methods `test()`, `exec()` and the `String` methods `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, `split()`. These methods provide a variety of regular expression operations, such as pattern matching, replacing certain characters, looking for substrings, splitting substrings, and so on [1].

2.2 ReDoS

Regular expressions are very useful in programming, but can be exploited to cause a regular expression denial-of-service attack (ReDoS). A ReDoS attack exploits the fact that some RegEx patterns can take a long time to evaluate, causing a service to be disrupted. The reason it can get stuck is that RegEx usually uses a naive backtracking algorithm that has a time complexity of $O(n^2)$ when trying to evaluate all possible states from the input string [2].

There are many "evil" regular expressions, one such RegEx is `(a+)+$`. When it

is evaluated with a string like `aaaaaaaaaaaaaaaaaX`, it will cause a DoS. The length of the string will determine how long the DoS will go on. One way to cause such a ReDoS attack is by RegEx injection, which is when an attacker controls two different inputs that will get matched somehow. The malicious user can then provide both the evil RegEx and the malicious string to cause the DoS [2].

2.3 SQL

Structured Query Language (SQL) is a language used to communicate with relational databases. It is commonly used in web applications to store and retrieve information such as user accounts, messages, and application data [3]. SQL queries can be constructed dynamically using strings, which is especially useful when queries need to be adapted based on user-provided data. For example, an application may want to retrieve all information related to a specific user:

```
"SELECT * FROM users WHERE userName='" + userName + "'"
```

where `userName` is a string variable representing the username of a certain user.

2.4 SQL Injection

SQL injection is a vulnerability that occurs when user-controlled input is incorporated into SQL queries without proper security measures. An attacker may exploit this by injecting malicious SQL code into a query, potentially allowing them to bypass authentication, retrieve sensitive information, modify database contents, or execute administrative operations on the database [4].

The SQL example above would be susceptible to such an attack if the `userName` variable is directly controlled by the user. An attacker could then provide an input such as `"' OR 'a'='a'-- "`, which would result in the following query:

```
"SELECT * FROM users WHERE userName=' ' OR 'a'='a'-- ' "
```

Since the condition `'a'='a'` always evaluates to true, the query would return every row in the `users` table.

2.5 CodeQL

Code Query Language (CodeQL) is developed by GitHub as a way to discover security vulnerabilities inside software projects. It can be used by integrating it into a CI/CD pipeline, or by individually analysing a project. CodeQL works by translating the entire codebase into a database and then evaluating queries against that database to identify patterns relevant to the analysis being performed [5].

There are two different ways to create the database, either through GitHub or

by letting CodeQL do it locally on your machine, with both methods resulting in the same type of database. The key difference is that GitHub can only do it if code analysis has been explicitly enabled for the repository, whereas local analysis can be performed whenever the source code is available. It works a bit differently depending on whether the project is built through an interpreted or a compiled language. Since JavaScript is an interpreted language, the project does not have to be compiled to create the database [5].

The queries written with CodeQL are similar to other query languages, such as SQL, but they operate on program elements represented by language-specific libraries rather than conventional database tables. [6]. There are prebuilt queries looking for common vulnerabilities hosted by GitHub, but it is also possible to build your own queries [7].

2.5.1 Sources and Sinks

Sources and sinks are core concepts in taint analysis. A source is a point where untrusted user-controlled input enters the program, and a sink is a location in the program where data ends up or is executed in some way [8]. Data flow from a source to a sink is an aspect of web applications that can cause insecure properties. If objects or input can move from a source to a sink, an attacker can potentially send malicious information that can be used as an attack vector [9].

To track flow from sources to sinks using CodeQL, paths are constructed by importing flows inside the query. With this, a query can be constructed to find when there is an insecure flow between these, and thus that flow can be broken [10].

2.6 Multi-repository variant analysis

To analyse many repositories simultaneously, GitHub provides a feature called multi-repository variant analysis (MRVA). The repositories to be analysed are specified in a JSON configuration file, which is used as input to the MRVA process. CodeQL then automatically generates databases for the selected repositories and executes the query remotely, allowing resources to be spared from the querying user. The main issue with this is that if public repositories are analysed, code scanning must be enabled and it must also be hosted on GitHub [11].

3 Goal

The goal of this project is to identify potential ReDoS and SQL injection vulnerabilities in open-source JavaScript and TypeScript GitHub repositories using CodeQL. The analysis is conducted using custom CodeQL queries that search for the following patterns:

- ReDoS: a user-controlled string evaluated against a user-controlled regular expression, or vice versa
- SQL injection: a database query constructed from user-controlled input

Any findings in a repository will be manually analysed to determine whether they are real vulnerabilities or false positives.

4 Description of work

This chapter describes the method used to analyse the different open-source projects.

4.1 ReDoS implementation

For this implementation, a query is created that would be able to find a flow of controlled user inputs in sources to sinks that would allow a malicious user to execute a ReDoS attack by providing a malicious string and an evil RegEx. The Query that is created looks for two sources that will flow to a sink and follow the pattern: `string.StringMethod(regex)` or `regex.RegExpMethod(string)`.

GitHub is then searched for open-source JavaScript/TypeScript projects that contain keywords such as `website`, `nodeJS`, `backend`, `web-app`, `web-application`, `official site`, `frontend`. Each repository is then manually reviewed, and those that appear to be web applications and are therefore likely to contain regular expressions are added to the MRVA JSON configuration file. Repositories are categorised by popularity, where popular repositories have more than 500 stars, and less popular repositories have between 50 and 500 stars.

The query is then run against the selected repositories using MRVA. If there is a hit, the code from that repository is manually analysed to see if it contains a flow vulnerable to ReDoS. If such a vulnerability is detected, it is reported to the owner of the repository.

4.2 SQL Injection implementation

To discover SQL injection vulnerabilities, a query similar to the one used for ReDoS is implemented. The query identifies source-to-sink data flows in which user-controlled input reaches database methods that execute SQL statements. 50 popular and 50 less-popular repositories are then selected for this analysis. These were chosen by searching GitHub using keywords such as `sql`, `mysql`, `pg`, `postgres`, `mssql`, `sqlite`, `knex`, `sequelize`. In addition to these, the OWASP Juice Shop repository (an educational repository with known vulnerabilities) is used to verify that the query works as intended. The SQL injection query is then run against these repositories, and any identified flows are examined to determine whether they are vulnerable to SQL injection attacks.

4.3 How to run

All project files can be found in the GitHub repository [RasmusSjo/DD2525-redos-sqli-codeql-analysis](https://github.com/RasmusSjo/DD2525-redos-sqli-codeql-analysis). The `databases.json` file is a copy of the configuration file containing all repositories that were analysed. In `queries/`, `ReDoS.ql` contains the query for detecting ReDoS and `SqlInjection.ql` contains the query for SQL injection. Lastly, `results/` contains the results of the variant analysis.

To run the code, set up CodeQL in VSCode as an extension. Add the queries as JavaScript queries and add the content of the `databases.json` file to the variant analysis configuration file. Then set up a public repository with at least one commit that is linked to the variant analysis. Lastly, run the query against the variant analysis repositories, and it will produce its result.

4.4 Explanation of code

4.4.1 The ReDoS query

The query for ReDoS sets up the function to check for sources and sinks. A source is basically anything that is untrusted: URL parameters, query strings, headers, etc. The sink function looks for all the dangerous methods (`match`, `search`, `replace`, `replaceAll`, `split`, `test`, `exec`) and that they follow the pattern explained in section 4.1. The code uses the `from` and `where` clauses to search for two flows: one flow from RegEX pattern source to RegEX pattern sink, and a flow from string source to string sink. The last part makes sure that it matches either of the wanted patterns and then selects those nodes if they exist.

4.4.2 SQL Injection query

For the SQL injection query, sources are defined in the same way as for the ReDoS query, i.e. any remotely controlled input is considered a source. The sink is defined as the argument to any of the following database methods: `query`, `execute`, `raw`, `prepare`, `run`, `all`, `$queryRaw`, `$queryRawUnsafe`, `$executeRaw` and `$executeRawUnsafe`. The query then checks for any flow from a source to a sink, and if such a flow exists, it is flagged as a potential vulnerability.

5 Results

The queries were each run against 50 popular repositories and 50 less popular repositories, with one additional control repository to verify correct behaviour.

All the repositories that were tested can be found in the `databases.json` file in the GitHub repository [RasmusSjo/DD2525-redos-sqli-codeql-analysis](https://github.com/RasmusSjo/DD2525-redos-sqli-codeql-analysis).

5.1 ReDoS

Of the 100 repositories analysed, 10 of the popular and 21 of the less popular had not enabled code analysis for their GitHub repositories, and could not be analysed using multi-repository variant analysis. For those that could be analysed, the ReDoS query found two less-popular repositories and three popular ones that had potential vulnerabilities (in addition to the control repository, that is from the labs). These were:

- apache/dolphinscheduler-website
- department-of-veterans-affairs/vets-website
- advplyr/audiobookshelf
- photo/frontend
- NaiboWang/EasySpider

5.1.1 Dolphinscheduler website

These code snippets in the website in the file `public/404.html` was identified as potential risks by the query. As it is seen the `replace()` method is called on the user controlled RegEx. The reason that this will not cause a ReDoS attack is that the method is different from the string version of `replace()`. It will only replace the current document with the provided URL.

`public/404.html`

```
38 window.location.replace(  
39   `/${lang}/blog${matchedName ? "/" + matchedName[1] : ""}`  
40 );
```

`public/404.html`

```
47 window.location.replace(url);
```

5.1.2 Vets website

The veteran website was under migration so the project was locked and could therefore not be tested manually to see if the vulnerability worked. Analysis of the code shows that, just like the dolphin scheduler, the query found a potential flow, but it was to a location replace instead of the string method replace. This is thus not vulnerable to ReDoS. The code is found in `src/applications/accredited../containers/AuthCallbackHandler.jsx`.

```
186 window.location.replace(to);
```

5.1.3 Audiobookshelf

This repository had 35 different alerts of potential ReDoS weaknesses but it is actually only 3 lines of code. It alerts different ways to find the flow.

server/controllers/LibraryController.js

```
626 const filterByGroup = payload.filterBy?.split('.').shift()  
627 const filterByValue = filterByGroup ? libraryFilters.decode(  
    payload.filterBy.replace(`${filterByGroup}.`, '')) : null
```

This part of the code, specifically row 627 since the replace method call upon filterByGroup as a literal string, meaning that even if it is an evil RegEx it will never be evaluated as such.

server/finders/BookFinder.js

```
290 const authorRe = new RegExp(`(^| | by |)${escapeRegExp(this.  
    cleanAuthor)}(?:= |$)`, 'g')  
291 const authorCleanedTitle = cleanAuthorForCompares(title)  
292 const authorCleanedTitleWithoutAuthor = authorCleanedTitle.  
    replace(authorRe, '')
```

In this code snippet, line 292 is the "dangerous" part. In line 292, the authorRe variable is cleaned with the escapeRegExp method which will add backslashes to all special characters meaning that it will not be evaluated as a dangerous RegEx. This part of the code can thus not cause a ReDoS.

server/libs/nodeCron/convert-expression/range-conversion.js

```
18 return expression.replace(new RegExp(text, 'i'), numbers.  
    join());
```

In this line, the program could be targeted by a ReDoS attack since replace executes a replace of a RegEx. The reason that this is also not a vulnerability is that the method this line of code is in, replaceWithRange(expression, text, init, end), is called from another method that sanitises text so that it can only be on the form "digit-digit", meaning that a malicious RegEx will not be executed in the replace method.

5.1.4 Photo Frontend

This program only gets alerts that are the same as the veteran website and dolphinscheduler, that is the query gets confused by the method replace for URL instead of string. There is thus no ReDoS vulnerability in this part of the codebase.

src/html/assets/javascripts/releases/4.0.2/tbx.js

```

1657 if (replace) {
1658     var href = location.href.replace(/(javascript:|#).*$/, '');
1659     location.replace(href + '#' + fragment);

```

5.1.5 EasySpider

In this first code snippet, it is yet again the URL `replace` method being flagged and not the string `replace` method, meaning that it is not vulnerable to a ReDoS attack.

ElectronJS/src/taskGrid/logic.js

```

661 let currentUrl = window.location.href;
662 let id = getUrlParam("id");
663 let newUrl = currentUrl.replace("id=" + id, "id=" + result +
    "&saveAs=1");

```

This could also have been an issue for a ReDoS attack if on line 776, the result of `match[0]` was wrapped inside a `RegExp` constructor. The `replace` method would then execute it as a `RegExp` but in this scenario it will only execute it as a literal string, meaning that it can not be used for ReDoS.

ElectronJS/main.js

```

774 let jsReplacedText = await execute_js(match[1], null, 0);
775 // 替换匹配到的 JS 语句
776 keyInfo = keyInfo.replace(match[0], jsReplacedText.toString());

```

5.2 SQL Injection

For the SQL injection query, most of the popular repositories had code scanning enabled; only four projects had it turned off. For the less-popular repositories, this number was instead 31, resulting in only 20 repositories being analysed. It did find some potential vulnerabilities, however. In addition to the control repository, it found three vulnerabilities in the popular repositories, and one among the less popular ones. The following repositories are those for which it found vulnerabilities:

- juice-shop/juice-shop (control repository)
- thevahidal/soul
- nalgeon/sqlime
- ujjwalguptaofficial/JsStore
- yzua/sqlite-online

5.2.1 juice-shop

This repository is an educational repository that contains intentional vulnerabilities, including SQL injection vulnerabilities. The query found three such vulnerabilities. Two of these had the same sink, located in `routes/login.ts`, and therefore only one will be shown here.

`routes/login.ts`

```
34 models.sequelize.query(`SELECT * FROM Users WHERE email = '${
    req.body.email || ''}' AND password = '${security.hash(
    req.body.password || '')}' AND deletedAt IS NULL`, {
    model: UserModel, plain: true })
```

`routes/search.ts`

```
23 models.sequelize.query(`SELECT * FROM Products WHERE ((name
    LIKE '%${criteria}%' OR description LIKE '%${criteria}%')
    AND deletedAt IS NULL) ORDER BY name`)
```

Both of these queries are vulnerable to SQL injection attacks since they use string interpolation with user-controlled input. For example, in the first vulnerability, an input such as `' OR 'a'='a';--` in the email field could bypass the intended email and password checks by making the `WHERE` condition always true and commenting out the remaining query. Now, since this is an educational project with intentional vulnerabilities, these findings are not reported.

5.2.2 soul

There were a total of 30 findings in this project, many of which shared the same sink and merely represented different paths to that sink. A commonality among all findings is that they interpolate user-controlled input into raw SQL/SQLite statements. Given the number of findings, not all of them will be analysed in depth. However, the findings can generally be categorised into two groups: statements using standard SQL operations such as `SELECT`, `UPDATE`, and `DELETE`, and statements using the SQLite-specific `PRAGMA` command.

`src/controllers/tables.js`

```
242 const { name: tableName } = req.params;
243 const query = `DROP TABLE ${tableName}`;
244 try {
245     const data = db.prepare(query).run();
```

In this snippet, the source is on line 242, where the `tableName` parameter is extracted from `req.params`. The sink is the `query` parameter on line 243, and from line 243, it shows that `tableName` is interpolated directly into the SQL statement, without parameterisation or escaping. This is a potential SQL injection vulnerability, and as such, the code is manually analysed.

This reported code snippet is part of the `deleteTable` method, which is exposed in the following way:

`src/routes/tables.js`

```
33 router.delete(  
34  ('/:name',  
35   hasAccess,  
36   validator(schema.deleteTable),  
37   controllers.deleteTable,  
38 );
```

From this snippet, we observe two relevant safeguards. First, the endpoint uses the `hasAccess` method to perform authorisation checks. Inspecting this method shows that authorisation is checked against the exact table name supplied by the user. Consequently, a non-superuser would generally only be able to delete tables for which they already possess permissions.

Second, the table name parameter is validated using a Joi schema before reaching the SQL sink. More specifically, the regular expression `/^[\\w-]+$/` restricts the input to alphanumeric characters, underscores, and hyphens, in addition to enforcing length constraints. This prevents characters such as whitespace, quotes, and semicolons from reaching the query. As a result, a potential attacker cannot alter the structure of the generated SQL statement, consequently prohibiting SQL injection for this endpoint.

Furthermore, analysis of the remaining reported paths shows that the same validation and authorisation mechanisms are used there. Hence, even if the source-to-sink flows found by the query appear to be vulnerable, the safeguards prevent exploitation. As a result, these findings are classified as false positives.

5.2.3 `sqlime`

Three vulnerable paths were identified in this project. All three had the same source, where two of the paths were subpaths of the third.

`js/locator.js`

```
5 // path creates a database path from the window location.  
6 function path() {  
7   return new DatabasePath(window.location.hash.slice(1));  
8 }
```

Focusing on the longest path, we see that user-controlled input from the URL eventually reaches a sink. Without context, this looks like a real SQL injection issue. However, `sqlime` is a sandboxed SQLite playground that lets users write and execute arbitrary SQL snippets. The input is therefore not injected into a trusted application query, but is the actual SQL program to execute. Since

execution is limited to the playground database, this finding is classified as a false positive and is not analysed further.

5.2.4 JsStore

One flow was flagged in this project:

```
src/worker/index.ts
5 if (IS_WORKER) {
6     const manager = new QueryManager();
7     (self as any).onmessage = function (e) {
8         manager.run(e.data);
9     };
10 }
```

The sink in this flow is the call to the `run` method, and the source is `e.data`. Inspection of the implementation shows that this is a false positive. The flagged `run` method is an application-specific method that does not execute SQL queries. As a result, the reported flow does not reach a SQL injection sink.

5.2.5 sqlite-online

A total of 9 flows were flagged in this project, all of which were determined to be false positives after manual inspection. In each case, the user-controlled input is sanitised before being incorporated into a query, preventing it from being interpreted as executable SQL.

One interesting thing is that the predefined SQL injection query provided by GitHub did not report any findings for this project, showcasing that it understands that the input is sanitised. The difference in results is likely due to our custom query not considering sanitisation routines, causing taint to be propagated even after the input has been sanitised. Another possibility is that our sink definition is more permissive than that of the predefined query. In either way, our approach over-approximates potential vulnerabilities, resulting in a higher number of false positives. This highlights the importance of both precisely defined queries and manual verification when interpreting the results.

6 Discussion

Several aspects of this project are interesting but could have been improved to produce even better results, and could potentially be used to help secure open-source software. For both our investigations, we could have investigated more projects, but also made sure that the selected projects were investigated more thoroughly. If we had more projects, the likelihood of finding an actual vulnerability would be higher, especially if those projects were less popular.

To make the analysis of the projects more thorough, we should have done two things differently. First, when choosing repositories to analyse, we did not spend much time looking at their code and mostly considered whether they matched our keywords or not. As a result, many of the selected repositories did not actually contain any regular expressions (for ReDoS) or SQL statements (for SQL injection). Since repositories lacking these constructs cannot contain the vulnerabilities being investigated, they are irrelevant to the analysis. As such, more time should have been spent on verifying that the repositories contained the relevant functionality.

The second improvement would be to also analyse the repositories that did not have code scanning enabled. In addition to improving coverage, this could have yielded more results. This is especially true for the less popular repositories, where 52 repositories across the two vulnerability categories had code scanning disabled. This is despite defining less popular repositories as those with fewer than 500 stars, a relatively high threshold that was chosen in the hope that these repositories would still have code scanning enabled.

Furthermore, since code scanning is disabled by default, projects that have it enabled may be more security-aware and could therefore be less likely to contain vulnerabilities. As a consequence, by analysing the repositories that had it disabled, we may have found more vulnerabilities, and especially, real vulnerabilities.

6.1 Query Improvements

The results show that the implemented taint-analysis queries were effective at finding cases where user-controlled input reaches security-vulnerable operations. At the same time, the analysis demonstrated an important limitation, which is that a detected flow does not imply an exploitable vulnerability. In the analysed repositories, several surrounding mitigations reduced exploitability, such as authorisation checks and sanitisation/validation of user input. As a result, manual inspection was required to evaluate the actual security impact of the findings, and in most cases, determine the findings to be false positives.

To reduce the number of false positives, the queries could be extended to check whether the sources are altered or sanitised before reaching the sink. In such cases, it could stop propagating taint. One more improvement would be to ensure only the correct sink methods are reported. This would have eliminated false positives such as the wrong `replace()` method. This could have been done by making sure that all the `String` methods were called from strings and the `RegExp` methods from `RegExp`.

A more specific improvement that could be made to the ReDoS query would be to look for more patterns. The pattern investigated in this project was very specific, and hence, less likely to occur. Had we also investigated more general

patterns, for example where a string is matched against a statically defined RegEx, the query may have yielded more results.

6.2 Conclusion

To conclude, this project demonstrated how CodeQL can be used to perform large-scale security analyses of open-source JavaScript and TypeScript repositories. Custom taint-analysis queries were developed to identify potential ReDoS and SQL injection vulnerabilities, and the resulting findings were manually inspected to determine their validity.

The results show that CodeQL is effective at identifying potentially vulnerable source-to-sink flows, but also highlight the importance of manual verification, as many reported findings were determined to be false positives. The study further showed that query precision and repository selection have a significant impact on the quality of the results. Future work could therefore focus on improving the precision of the queries and refining the repository selection process to increase the likelihood of finding relevant vulnerabilities.

References

- [1] MDN Web Docs. *Regular expressions - JavaScript | MDN*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions. Accessed: 2026-05-24. 2026.
- [2] OWASP Foundation. *Regular expression Denial of Service - ReDoS*. 2026. URL: https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS (visited on 05/24/2026).
- [3] AWS. *What is SQL (Structured Query Language)?* URL: <https://aws.amazon.com/what-is/sql/> (visited on 05/27/2026).
- [4] OWASP Foundation. *SQL Injection*. URL: https://owasp.org/www-community/attacks/SQL_Injection (visited on 05/27/2026).
- [5] GitHub. *About CodeQL*. 2026. URL: <https://codeql.github.com/docs/codeql-overview/about-codeql/> (visited on 05/24/2026).
- [6] GitHub. *CodeQL library for JavaScript*. 2026. URL: <https://codeql.github.com/docs/codeql-language-guides/codeql-library-for-javascript/#codeql-library-for-javascript> (visited on 05/24/2026).
- [7] GitHub. *About CodeQL queries*. 2026. URL: <https://codeql.github.com/docs/writing-codeql-queries/about-codeql-queries/#about-codeql-queries> (visited on 05/24/2026).
- [8] Kaustubh Rai. *Understanding Sources and Sinks: A Guide to Taint Analysis*. July 12, 2024. URL: <https://breachforce.net/source-and-sinks> (visited on 06/02/2026).
- [9] Musard Balliu. “Web Application Security, Part 1”. Lecture in course DD2525 Language-Based Security, KTH Royal Institute of Technology. Lecture notes/Slides. Apr. 13, 2026.
- [10] GitHub. *Creating CodeQL path queries*. 2026. URL: <https://codeql.github.com/docs/writing-codeql-queries/creating-path-queries/> (visited on 05/24/2026).
- [11] GitHub. *Running CodeQL queries at scale with multi-repository variant analysis*. 2026. URL: <https://docs.github.com/en/code-security/how-tos/find-and-fix-code-vulnerabilities/scan-from-vs-code/running-codeql-queries-at-scale-with-multi-repository-variant-analysis> (visited on 05/24/2026).

A Contributions

We began by researching CodeQL, how it works and how we can use it, as well as how we would select repositories to analyse.

After this, we created the queries, selected repositories and analysed the results. This was done individually for one vulnerability, where Jonathan focused on ReDoS and Rasmus focused on SQL injection.

Lastly, we discussed our findings and started to work on the report. We wrote the report together, but each of us wrote about our own part individually.

B AI usage

AI tools were used to help with LaTeX formatting and to improve the language in the report (grammar, spelling, citations). It was also used when writing the README in the GitHub repository.

C GitHub Repository

[RasmusSjo/DD2525-redos-sqli-codeql-analysis](https://github.com/RasmusSjo/DD2525-redos-sqli-codeql-analysis).